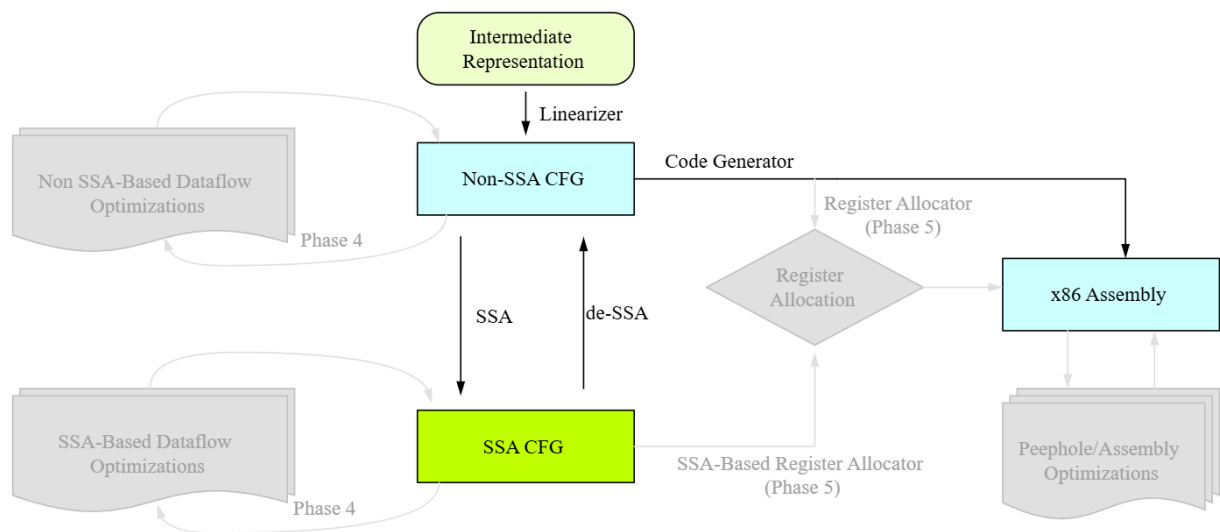


Теория языков программирования и методы трансляции

Рекомендации по разработке компилятора

Проект разработки компилятора — один из немногих проектов в области разработки ПО, который начинается со спецификации и заканчивается большим программным продуктом. Существует множество проектных решений, которые вы должны принять на этом пути, и существует бесконечное количество способов приступить к разработке вашего компилятора.

Этап 3

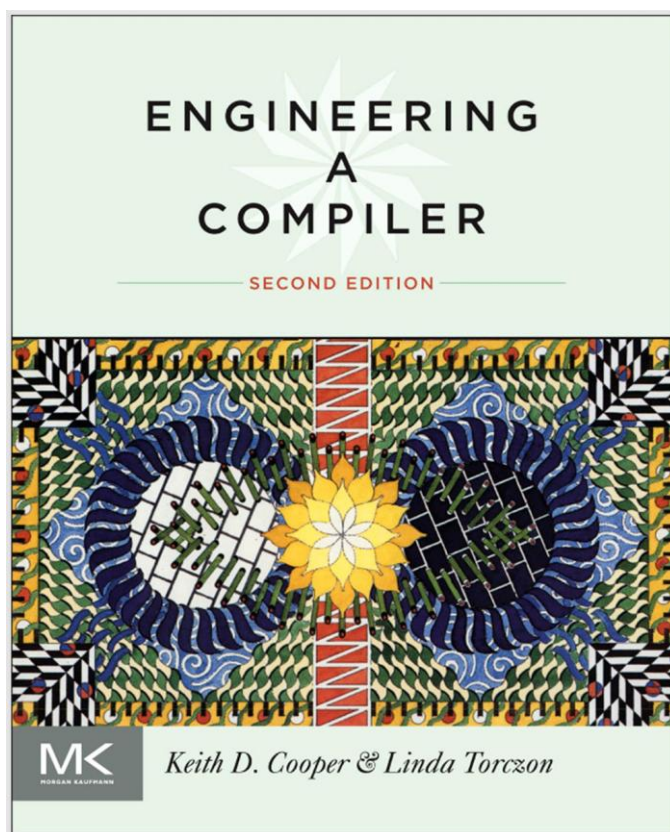


К концу 2 этапа у вас должно быть высокоуровневое промежуточное представление вашей программы, которое (предположительно) семантически корректно. Цель 3 этапа — сгенерировать правильный код x86. Ожидается, что ваш код может работать очень медленно, но вторую половину занятия мы посвятим оптимизации сгенерированного кода x86.

Линеаризация

Это часть вашего компилятора, которая преобразует сложные выражения в высокоуровневом промежуточном представлении в более простые линейные инструкции с тремя адресами. На этом этапе поток управления также должен быть преобразован в переходы. Существует множество способов сделать это, и выбор зависит от структуры вашего высокоуровневого промежуточного представления. Мы можем порекомендовать вам некоторые из следующих ресурсов:

- Copper et al, *Engineering a Compiler*, Глава 4 содержит подробное описание того, как можно спроектировать контекстно-свободную грамматику, а также рекомендуемые структуры данных



Aho et al, (Dragon Book) Compilers Principles Techniques and Tools, Глава 4 содержит подробное описание того, как представить многие синтаксические структуры в контекстно-свободной грамматике, включая некоторые распространенные языковые функции, которых нет в Desaf. Упрощенная версия контекстно-свободной грамматики из книги может оказаться полезной при проектировании.



SSA (Static Single Assignment) статическое одиночное присваивание

Одно из самых важных решений для вашей команды на этапах 3–5 — использовать или не использовать форму статического одиночного присваивания (Static Single Assignment, SSA). SSA использовать необязательно, но в целом самые эффективные компиляторы используют эту форму.

Основная причина, по которой SSA полезна, заключается в том, что она позволяет более эффективно анализировать потоки данных, что, в свою очередь, повышает эффективность оптимизации на этапах 4 и 5. Однако это усложняет реализацию кода. В частности, необходимо написать проход для преобразования графа потока команд в SSA-форму, а затем еще один проход для преобразования из SSA-формы (так называемое обратное преобразование SSA).

Что касается рабочего процесса, то у команд, использующих SSA, на третьем этапе будет больше работы, а на четвертом — меньше, поскольку SSA значительно упрощает оптимизацию потоков данных. С другой стороны, у команд, которые не используют SSA, на третьем этапе будет меньше работы, а на четвертом — больше.

Преобразование в SSA-форму

Преобразование графа потока управления в SSA-форму предполагает, что каждой переменной присваивается значение только один раз. Из-за потока

управления нам также приходится использовать узлы ϕ для разрешения некоторых переменных. Ключевой вопрос, который необходимо решить при преобразовании в SSA-форму, — это *оптимальное размещение узлов ϕ* . На эту тему написано множество научных статей, а также учебников, в которых представлена каноническая версия этого процесса. Мы рекомендуем следующие ресурсы:

- *Copper et al, Engineering a Compiler*, в глава 9.3, очень хорошо объясняется, как составить форму SSA.
- [SSA-based Compiler Design \(The SSA Book\)](#): Глава 3.1, также объясняют, как создать SSA-форму.

Статья, в которой описывается, как преобразовать в форму SSA из *non-SSA CFG* Cytron et al (1991), [Efficiently Computing Static Single Assignment Form and the Control Dependence Graph](#).

способы прямого преобразования высокоуровневого IR в форму SSA. Канонической работой в этой области является:

Braun et al (2013), [Simple and Efficient Construction of Static Single Assignment Form](#) — который используется в компиляторе Go и OpenJDK.

Возможно, вам также будет полезна эта статья о границах доминирования:

Copper et al (2006), *A Simple, Fast Dominance Algorithm*

Преобразование из SSA-формы (de-SSA)

Перед генерацией кода необходимо выполнить преобразование из формы SSA, поскольку у узлов ϕ нет аналогов в машинном коде. Следовательно, для перевода программы в код x86 узлы ϕ нужно преобразовать в операции копирования. Ключевая сложность на этом этапе заключается в решении проблем «потерянного копирования» (lost copy) и «обмена» (swap), возникающих при наличии циклов в графе потока управления (CFG). Мы рекомендуем следующие материалы:

- *Copper et al, Engineering a Compiler*, в глава 9.3.5, приводится ясное описание проблем «потерянного копирования» и «обмена», а также алгоритм их решения, который теоретически можно реализовать на практике.
- [SSA-based Compiler Design \(The SSA Book\)](#): Глава 3.2 предлагает псевдокод алгоритмов для корректного устранения формы SSA, однако не дает подробного разъяснения самих проблем; поэтому перед анализом кода из книги по SSA рекомендуется ознакомиться с книгой Купера.

Академическая статья по де-SSA:

- Boissinot et al (2009), [Revisiting Out-of-SSA Translation for Correctness, Code Quality, and Efficiency](#)

Гибридные представления графа потока управления (CFG)

Вы можете решить объединить представления CFG, использующие форму SSA и не использующие её. Этого можно добиться, если включить в представление CFG узлы phi. При линеаризации высокоуровневого промежуточного представления (IR) эти узлы phi можно вставить в граф, а перед генерацией кода для x86 — выполнить еще один проход по CFG, чтобы удалить их и передать граф генератору кода.

Некоторые преимущества использования гибридного IR:

- Меньше дублирования кода, так как значительная часть представлений CFG (с SSA и без) будет совпадать

- Архитектурное сходство с LLVM
- Приятное чувство собственной компетентности :)

Некоторые недостатки:

- Значительно сложнее выполнять отладку, поскольку реализация CFG оказывается тесно связанной с формой SSA

- Труднее определить, что именно работает некорректно: проходы SSA или сам CFG

- Необходимо отслеживать текущее состояние CFG (содержит ли он узлы phi) и выполнять соответствующие оптимизации

Генератор кода

Генератор кода принимает на вход граф потока управления (CFG), не приведенный к форме SSA, и преобразует его в код на языке ассемблера x86. При проектировании CFG стоит учитывать аспекты, упрощающие последующую генерацию кода. Вот несколько рекомендаций:

Постарайтесь сделать так, чтобы с генерируемым ассемблерным кодом было удобно работать программно (то есть не выводите код в виде обычных строк), поскольку на более поздних этапах может потребоваться его оптимизация.

Если вы все же решите использовать форму SSA, убедитесь, что генератор кода работает корректно независимо от того, преобразован ли CFG в SSA или нет. Это облегчит отладку.

На третьем этапе сосредоточьтесь на корректности работы программы, пока не уделяя внимания скорости выполнения. Проще всего этого добиться, активно используя стек для хранения переменных.

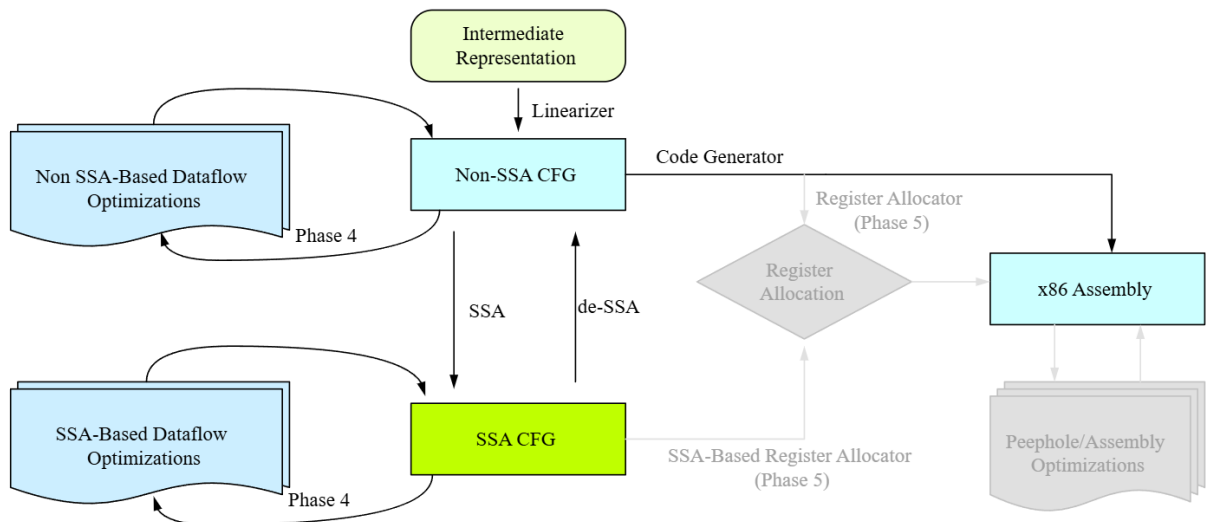
Вот еще несколько полезных ресурсов, к которым можно обратиться:

- *Aho et al, (Dragon Book) Compilers Principles Techniques and Tools* в 8 главе подробно описан этап генерации кода. В этой главе рассматриваются некоторые методы оптимизации — например, оптимизация в окне (peephole optimization) и выбор

инструкций, — реализацию которых, возможно, стоит отложить до более поздних этапов.

- Материалы по **архитектуре x86** (Литература, ресурсы, статьи).
- **Godbolt** — сервис, позволяющий изучить, как различные компиляторы генерируют код на языке ассемблера, и почерпнуть из этого полезные идеи.

Этап 4



На этапе 4 вы реализуете несколько оптимизаций потока данных.

Глобальное распространение копий (Global Copy Propagation)

Цель распространения копий — сократить количество операций копирования переменных и, что еще важнее, **выявить возможности для других оптимизаций** (например, последовательность $x = a + b; c = b; y = a + c$ превращается в $x = a + b; c = b; y = a + b$, что позволяет применить исключение общих подвыражений).

- При использовании SSA-представления проверка допустимости, описанная выше, выполняется автоматически благодаря свойству единственного присваивания. Однако есть несколько нюансов, требующих внимания:

- **Phi-узлы с идентичными аргументами — это скрытые операции копирования.** После распространения некоторых копий могут появиться phi-узлы, у которых все аргументы одинаковы, например $x_2 = \text{phi}(x_1, x_1, x_1)$. Это новая копия! В идеале ваш алгоритм должен продолжить работу и заменить все вхождения x_2 на x_1 .

- **Проблема «потерянной копии».** Распространение копий усложняет процесс выхода из SSA-формы. Пример этой проблемы можно найти на данных слайдах. Кратко: остерегайтесь **критических ребер** (ребер $A \rightarrow B$, где у узла A несколько преемников, а у узла B — несколько предшественников) и устраняйте их путем вставки пустых блоков (заменяйте $A \rightarrow B$ на $A \rightarrow C \rightarrow B$, где C — пустой блок).

Об этой проблеме необходимо знать, иначе вы рискуете потратить дни, безуспешно пытаясь разобраться в ситуации.

- Ваш алгоритм может сначала находить копию `x = y` и заменять все вхождения `x` на `y`, но это требует полного обхода графа потока управления (CFG) для каждой копии! Альтернативный подход — поддерживать отображение `orig`, инициализированное как `orig[x] = x`. Тогда при встрече с `x = y` нужно обновить `orig[x] = orig[y]` и по ходу выполнения заменять все вхождения `x` на `orig[x]`. При таком подходе один проход позволяет распространить множество копий, а также эффективно обрабатывать транзитивность. Мы рекомендуем обходить блоки в порядке **прямого обхода дерева доминаторов (pre-order)**: это гарантирует, что все случаи использования переменной `x` будут встречены после ее определения (в точке которого может обновляться значение `orig[x]`).

- Может оказаться, что одного полного обхода графа потока управления (CFG) недостаточно для распространения всех копий, как бы вы ни старались, — и это вполне допустимо. В таком случае просто выполняйте обходы до тех пор, пока очередной проход не перестанет вносить какие-либо изменения.

- В процессе распространения копий могут оставаться «мертвые» копии. Нет необходимости удалять их в рамках этого же этапа — эту задачу лучше оставить для этапа устранения мертвого кода!

Глобальное исключение общих подвыражений

- SSA снова упрощает задачу, поскольку нам не нужно беспокоиться о том, что переназначение переменных сделает выражение недоступным.

- На этом этапе вам понадобится карта, основанная на выражениях. Если предположить, что это хэш-карта, это означает, что вам нужно будет определить функции хеширования и равенства для дерева выражений. Разве это не утомительно? Хотя это правильный путь, проще всего написать функцию, которая отображает выражение в **строковое каноническое представление** и вместо этого заставляет вас сопоставлять эти строки с ключами. Это остается правильным, пока вы делаете это осторожно.

- У вас может возникнуть соблазн написать свой пропуск таким образом, чтобы учитывать коммутативность и ассоциативность или даже некоторые алгебраические тождества. **Сопровитляйтесь на мгновение этому повышению IQ и дважды подумайте о соотношении затрат и выгод, прежде чем идти в эту кроличью нору.** Сделать это эффективно (например, путем сопоставления эквивалентных выражений с одним и тем же хешем или каноническим выражением) еще сложнее. Реальность такова, что многие устранимые общие выражения дословно записаны в исходном коде, поэтому даже наивный подход может работать достаточно хорошо.

Глобальное устранение мертвого кода

- Использование SSA упрощает задачу и в этом случае. Удаление мертвого кода (DCE) для IR без SSA выполняется в обратном порядке, чтобы выявить бесполезные (пере)присваивания переменной. В SSA повторные присваивания переменных исходной программы уже разбиты на несколько определений переменных SSA. Поэтому DCE для SSA не обязательно выполнять строго в обратном порядке. Вместо этого можно использовать алгоритм на основе очереди задач (worklist):

1. Инициализируйте очередь задач всеми переменными, используемыми в терминаторах блоков (например, в условных переходах или операторах возврата);
2. Для каждой переменной из очереди:
 - a. отметьте ее как «живую» (используемую), и
 - b. добавьте в очередь все переменные, к которым происходит обращение в определении данной переменной (если они еще не добавлены);
3. Удалите все определения переменных, не отмеченные как «живые».

- Независимо от того, используете ли вы SSA, **следите за тем, чтобы не удалять инструкции с побочными эффектами**. К ним относятся вызовы **функций** и **операции записи в массив**. Разумеется, существуют исключения, когда такие инструкции можно удалить после анализа, но это относится к продвинутым методам оптимизации, которые лучше реализовывать в виде отдельных проходов компиляции.

- Вы также можете реализовать оптимизации, **удаляющие «мертвые» блоки, недостижимость которых можно доказать**. К ним относятся:

1. любой блок, к которому нет пути от начального блока в графе потока управления (CFG) — такие блоки могут остаться после предыдущих оптимизаций; или

2. любой блок, к которому ведет путь от начального блока в CFG, но **существует ребро, которое гарантированно не будет пройдено**. Например, переход, зависящий от константы, делает одну из ветвей перехода недостижимой. Впрочем, мертвые блоки встречаются гораздо реже, чем мертвые инструкции, и их удаление не является обязательным условием реализации DCE в рамках данного курса.

В каком порядке выполнять оптимизации?

На данном этапе вы, вероятно, понимаете, что определенная последовательность оптимизаций может быть более эффективной, чем другие. Например, удаление мертвого кода (DCE) лучше выполнять в конце, а не в начале,

чтобы оно могло устранить и тот «мусор», который возник в результате работы других оптимизаций. Однако всё не всегда так просто. Рассмотрим следующий случай:

- Распространение копий (copy propagation) открывает дополнительные возможности для исключения общих подвыражений (CSE), но
- CSE может заменять сложные выражения простыми копиями, создавая тем самым условия для повторного выполнения распространения копий.

Что должно идти первым? Ситуация усложняется по мере добавления в компилятор новых этапов (проходов) оптимизации. Определение оптимального порядка выполнения этих этапов остается предметом активных исследований. Для нашего учебного проекта разумной стратегией будет следующая:

1. Выбрать последовательность этапов, которая кажется логичной, но не пытаться довести её до идеала.
2. Выполнять этапы в выбранном порядке X раз, где X — небольшое целое число (например, 10).

Таким образом, каждый этап будет выполняться до и после всех остальных достаточное количество раз, что, как мы надеемся, позволит использовать все возможности для оптимизации. В реальных промышленных компиляторах такой подход не применяется, поскольку он может неоправданно замедлить компиляцию, однако в рамках данного курса время компиляции не является критическим фактором.

Недостаток такого «слепого» применения оптимизаций к промежуточному представлению (IR) заключается в том, что это усложняет отладку. Ошибочное преобразование на любом этапе может привести к некорректной работе компилятора. Поэтому мы даем следующий совет:

Общие принципы реализации оптимизации

Чтобы упростить себе задачу, настоятельно рекомендуем делать этап оптимизации (pass) **чистым, идемпотентным и детерминированным**.

- **Чистота** означает, что ваш этап оптимизации должен представлять собой преобразование IR в IR без каких-либо глобальных побочных эффектов. Это облегчает анализ, композицию и тестирование в изоляции.
- **Идемпотентность** означает, что этап оптимизации не должен вносить никаких изменений в IR, если не находит возможностей для оптимизации, на которые он рассчитан. Благодаря этому, если в процессе отладки выяснится, что конкретный этап не имеет отношения к устраняемой ошибке, его можно смело отключить, не опасаясь каскадных эффектов, способных усложнить отладку.
- **Детерминированность** означает, что при одних и тех же входных данных (IR) этап оптимизации должен выдавать идентичный результат (IR) при каждом запуске. В некоторых языках порядок итерации по элементам хеш-карты не является

детерминированным — будьте внимательны! Детерминированность значительно упрощает воспроизведение ошибок (и их выделение в регрессионные тесты).

Другие виды оптимизации

- **Распространение констант** (constant propagation): вычисление на этапе компиляции выражений, операнды которых являются константами времени компиляции.
- **Вынос инвариантного кода из цикла** (loop-invariant code motion / code hoisting): перемещение кода, инвариантного относительно цикла, из тела цикла в блок, предшествующий этому циклу. SSA не обязательно упрощает оптимизацию циклов — она просто меняет подход к ней. Возможно, вам будет интересно узнать, что для оптимизации циклов LLVM использует особую форму SSA — LCSSA. Подробности см. здесь.

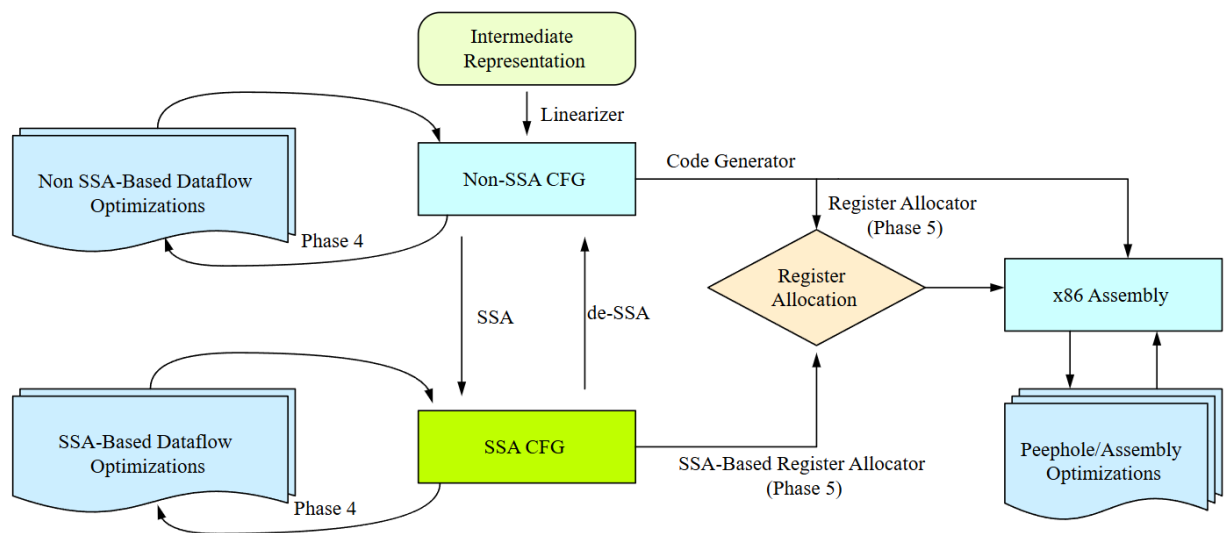
Инверсия цикла: преобразует цикл while в цикл do-while.

```
void pre_inversion() {
    while (/* condition */) {
        /* Loop body */
    }
}
// Gets converted to
void post_inversion() {
    if (/* condition */) {
        do {
            /* Loop body */
        } while (/* condition */);
    }
}
```

Напомним, что одним из аргументов против агрессивного выноса кода (hoisting) является снижение производительности в случае, если цикл ни разу не выполнится. При инверсии цикла выносимый код помещается в ветвь `if`, поэтому он выполняется лишь тогда, когда цикл срабатывает хотя бы один раз.

GVN-PRE (Global Value Numbering — Partial Redundancy Elimination) — это весьма мощный метод оптимизации, включающий в себя устранение общих подвыражений, вынос инвариантного для цикла кода (после инверсии цикла) и многое другое. Этот метод отличается концептуальной элегантностью, однако его реализация значительно сложнее.

Этап 5



На пятом этапе вам предстоит реализовать распределение регистров, а также другие оптимизации (например, локальные оптимизации — peephole optimizations) по вашему выбору.

Просто покажи мне код: примеры реализации IR на практике

Отличный способ изучить проектирование компиляторов — посмотреть, как это делают опытные разработчики. Ниже мы собрали список компиляторов с открытым исходным кодом, которые могут послужить ценным примером при разработке вашего собственного решения. Однако помните, что многие из этих компиляторов решают более масштабные задачи и работают со сложными языками. Не копируйте их архитектуру бездумно; подходите к делу взвешенно, чтобы избежать излишнего усложнения системы.

- **LLVM** — это фреймворк для создания бэкендов компиляторов, используемый в Clang (C/C++/Fortran), Rust, Swift и многих других языках; это настоящий кладезь оптимизаций, основанных на форме SSA (Static Single Assignment — статическое одиночное присваивание).
 - Бегло просмотрите [справочное руководство по языку](#) (Language Reference Manual), чтобы получить представление об SSA IR.
 - Если вы хотите узнать, как в LLVM реализована конкретная оптимизация, начните с чтения [руководства для программиста](#) (Programmer's Manual). Также можно попросить языковые модели (LLM) помочь вам разобраться в кодовой базе.
- **Craneflirt** — это бэкенд компилятора, написанный на Rust и используемый во многих реализациях WebAssembly. Если ваша команда использует Rust, кодовая база Craneflirt станет ценным примером того, как можно представить IR с помощью конструкций этого языка.
 - Ознакомьтесь со справочным руководством по IR ([IR Reference](#)).

- Для справки: в Cranelift используется разновидность SSA, называемая **«базовыми блоками с параметрами»** (basic blocks with parameters), что эквивалентно использованию phi-узлов. Также здесь применяется **граф потока данных** (data flow graph) для явного отслеживания зависимостей между инструкциями.
- Кодовая база Cranelift может показаться пугающей. Мы рекомендуем начать с изучения компонентов ``InstructionData``, ``Layout`` и ``DataFlowGraph``.
- **Компилятор языка Go** написан на самом Go и также использует IR на основе SSA. Примечательно, что в нем реализован алгоритм распределения регистров **методом линейного сканирования** (linear scan), работающий непосредственно поверх SSA. Если ваша команда рассматривает подобный подход, стоит изучить реализацию распределения регистров в Go.
 - В блоге Red Hat есть статья с общим описанием алгоритма распределения регистров в Go.
 - Код можно найти здесь; он занимает около 3000 строк (включая подробные комментарии) и вполне поддается пониманию при должном усердии.
- **libFirm** — это бэкенд компилятора, использующий графовое представление SSA; он был изначально разработан Себастьяном Хаком (Sebastian Hack) и содержит эталонную реализацию его алгоритма **распределения регистров методом раскраски графа**, работающего непосредственно поверх SSA. Как многие из вас позже убедятся на собственном опыте, прямое распределение регистров для кода в форме SSA — это крайне сложная и запутанная тема, а в соответствующей статье не хватает подробностей. И здесь на помощь приходит код libFirm.
 - Начните с изучения файлов bespill.c и bespillbelady.c.
- **QBE** — это бэкенд компилятора, написанный на языке C; его цель — «обеспечить 70% производительности промышленных компиляторов при объеме кода, составляющем лишь 10% от их размера». В нем реализованы многие ключевые оптимизации, однако автор намеренно сохраняет кодовую базу в масштабах любительского проекта, чтобы код было проще понять и модифицировать. Если исходный код упомянутых выше компиляторов кажется вам слишком сложным и пугающим, попробуйте QBE!